

Inteligencia Artificial

Estado del Arte: Problema de Programación de Conductores de Buses (*Bus Driver Scheduling Problem*, BDSP)

Vicente Luongo
ROL: 202073637-5

5 de octubre de 2026

Evaluación

Resumen (5 %):	_____
Introducción (5 %):	_____
Definición del Problema (10 %):	_____
Estado del Arte (40 %):	_____
Modelo Matemático (20 %):	_____
Conclusiones (15 %):	_____
Bibliografía (5 %):	_____
Nota Final (100 %):	_____

Resumen

El *Bus Driver Scheduling Problem* (BDSP) consiste en construir los turnos diarios de trabajo de los conductores de una empresa de transporte público, de modo que cada tramo de recorrido de los buses sea cubierto exactamente por un conductor, respetando la legislación laboral y los convenios colectivos. Se trata de un problema de optimización combinatoria NP-duro, cuyo costo impacta directamente en la operación de las empresas, dado que la mano de obra constituye una de sus principales partidas de gasto. En este documento se define el problema y la variante a estudiar, cuyo objetivo es minimizar el tiempo pagado total bajo reglas de descanso, sobretiempo, trabajo nocturno, tiempo mínimo pagado y duración máxima del turno. Se revisa la literatura desde los modelos clásicos de partición y cobertura de conjuntos hasta las metaheurísticas y los enfoques híbridos recientes, y se propone un modelo de programación lineal entera mixta que captura todas las reglas de la variante estudiada.

1. Introducción

La planificación de un sistema de transporte público por buses suele dividirse en varios sub-problemas encadenados: diseño de la red, construcción de itinerarios (*timetabling*), programación de vehículos (*vehicle scheduling*), programación de tripulaciones (*crew scheduling*) y asignación de turnos a personas concretas en un horizonte de semanas (*crew rostering*) [13, 17]. El BDSP

ocupa justamente la etapa de programación de tripulaciones: con el plan de vehículos ya fijado, hay que formar los turnos (*duties*) que cubran todo el trabajo de conducción de un día.

El interés por este problema es ante todo económico y legal. Los conductores representan uno de los componentes de mayor peso en el costo operacional de una empresa de transporte [25], y una asignación deficiente se traduce en horas extraordinarias, tiempos muertos pagados y más conductores de los necesarios. A la vez, los turnos deben cumplir reglas laborales estrictas (descansos mínimos, jornada máxima, recargos por trabajo nocturno), lo que convierte la construcción manual de turnos en una tarea lenta y difícil de optimizar. Además, el problema es NP-duro incluso en versiones muy simplificadas [9, 10], lo que justifica el uso de técnicas de inteligencia artificial e investigación de operaciones.

Este informe presenta el estado del arte del BDSP y formula un modelo matemático para la variante asignada en el curso, basada en las reglas de una empresa brasileña descritas por Constantino et al. [3]. La organización es la siguiente: la Sección 2 define el problema, su terminología, la variante estudiada y los problemas relacionados; la Sección 3 revisa los principales métodos de resolución propuestos en la literatura; la Sección 4 presenta dos modelos matemáticos, uno general y otro específico para la variante estudiada; y la Sección 5 expone las conclusiones del estudio hasta la última revisión realizada.

2. Definición del Problema

2.1. Terminología

El BDSP recibe como entrada la programación de vehículos, es decir, la secuencia de viajes que realiza cada bus durante el día (su *bloque*). A lo largo de cada bloque hay *puntos de relevo* (*relief opportunities*): pares lugar-instante en que un conductor puede dejar el vehículo y otro tomarlo. El trabajo entre dos puntos de relevo consecutivos se llama *tarea* o *tramo*, y es la unidad mínima e indivisible de trabajo que se asigna a un conductor [3]. Las paradas intermedias de un tramo no son relevantes, pues solo afectan su duración.

Un *turno* (*duty*) es la secuencia ordenada de tramos que realiza un conductor en un día. Se llama *spreadover* al tiempo transcurrido entre el inicio del primer tramo y el término del último. Entre dos tramos consecutivos de un turno puede haber una pausa, ya sea un *descanso* (no pagado, típicamente destinado a alimentación) o un *tiempo ocioso* (pagado). Una secuencia de tramos trabajados sin un descanso intermedio recibe el nombre de *stretch* [3, 17].

2.2. Descripción general

En términos generales, el BDSP busca un conjunto de turnos tal que:

- **Cobertura:** cada tramo es realizado por exactamente un conductor; no se permite que dos conductores compartan un tramo ni que un tramo quede sin cubrir.
- **Continuidad temporal y espacial:** un conductor solo puede comenzar un tramo si ya terminó el anterior y si se encuentra en el lugar donde este comienza. Sí puede cambiar de bus o de línea en cualquier punto de relevo.
- **Factibilidad laboral:** cada turno cumple las reglas de la legislación y de los convenios (descansos, jornada, sobretiempo, etc.).
- **Optimalidad:** se minimiza el costo total, usualmente medido como el tiempo pagado a los conductores, el número de conductores o una combinación de ambos.

Las *variables* del problema corresponden, por lo tanto, a qué conductor realiza cada tramo (lo que determina implícitamente la secuencia de cada turno), y las decisiones derivadas de ello: qué pausa se considera descanso, cuánto sobretiempo se genera, etc.

2.3. Variante estudiada

La variante asignada corresponde a la estudiada por Constantino et al. [3], con reglas provenientes de la legislación laboral brasileña y los convenios sindicales de una empresa de transporte urbano. Cada tramo queda descrito por su línea, punto de partida, punto de término, hora de partida, duración y bus. Las reglas, parametrizadas en cada instancia, son las siguientes (entre paréntesis, los valores del caso de referencia):

1. **Stretch máximo y descanso mínimo.** Un conductor no puede trabajar más de $T^{\text{máx}}$ (360 min) sin un descanso; si el turno excede ese tiempo, debe incluir un descanso de al menos $R^{\text{mín}}$ (90 min).
2. **Umbral de descanso pagado.** El descanso no se paga hasta un umbral R^{pag} (300 min); el exceso por sobre él se considera tiempo ocioso y se paga.
3. **Tiempo pagado mínimo.** Todo turno se paga por al menos $P^{\text{mín}}$ (432 min), aunque el trabajo efectivo sea menor.
4. **Sobretiempo.** El tiempo trabajado por sobre la jornada normal es sobretiempo, no puede superar $O^{\text{máx}}$ (120 min) y recibe un recargo de β_O (50 %).
5. **Trabajo nocturno.** La conducción entre las 22:00 y las 05:00 se corrige por un factor f_N (60/52, es decir, 52 minutos trabajados se cuentan como 60) y recibe un recargo de β_N (20 %).
6. **Duración máxima del turno.** El spreadover, incluyendo descansos no pagados, no puede superar $S^{\text{máx}}$ (780 min).
7. **Cambio de línea.** Los cambios de línea dentro de un turno están permitidos, pero se penalizan con p_L (1 min) cada uno.

El objetivo es minimizar el tiempo pagado total de todos los conductores más las penalizaciones por cambio de línea. Nótese que, a diferencia de otras variantes, el número de conductores no se minimiza explícitamente; sin embargo, la regla del tiempo pagado mínimo induce a usar pocos turnos bien aprovechados.

La Figura 1 muestra la instancia de ejemplo entregada en el curso, con seis tramos de dos líneas. Un único conductor no podría cubrirlos todos, pues el spreadover resultante (05:50–20:35, 885 min) excede $S^{\text{máx}}$. La solución natural usa dos conductores, uno por bus, cada uno con menos de 200 minutos de trabajo; por la regla de tiempo mínimo, cada uno se paga 432 minutos, lo que da un costo de 864. El ejemplo ilustra cómo el tiempo mínimo pagado y el tiempo ocioso dominan el costo cuando los turnos son cortos.

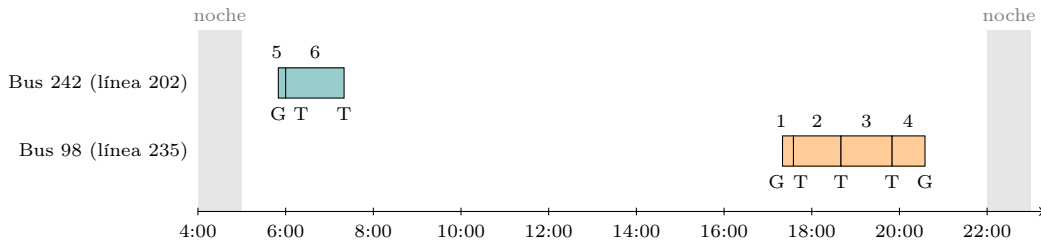


Figura 1: Instancia de ejemplo del curso: seis tramos sobre dos buses. Las letras indican los puntos de relevo (G: garaje, T: terminal) y las zonas grises, el horario nocturno.

2.4. Problemas relacionados y variantes

El BDSP forma parte de la familia de problemas de *crew scheduling*, a la que también pertenecen la programación de tripulaciones aéreas y ferroviarias; en todos ellos hay que cubrir tareas con secuencias factibles de trabajo, aunque las reglas y la escala cambian. Está muy ligado al *vehicle scheduling*, que determina los bloques, y al *crew rostering*, que encadena los turnos diarios en calendarios semanales o mensuales considerando aspectos de calidad de vida [13]. En el plano teórico, se relaciona con el *fixed job scheduling* con restricciones de spreadover y de tiempo de trabajo, cuya NP-dureza demostraron Fischetti et al. [9, 10].

Las variantes más conocidas del BDSP se diferencian en: (i) el *objetivo*, que puede ser minimizar el número de conductores (versión *unicost*) [27], el costo total [3], o ambos, incluso de forma multiobjetivo [21]; (ii) el *alcance*, pues algunas versiones resuelven la programación de vehículos y conductores de forma integrada [12]; (iii) la existencia de uno o varios *depósitos*, con la exigencia de terminar el turno en el depósito de partida [17]; (iv) la separación por línea frente a un enfoque de red, en que un conductor puede cambiar de línea [23, 3]; y (v) la complejidad de las reglas de pausas, como en el caso austriaco estudiado por Kletzander y Musliu [15].

3. Estado del Arte

3.1. Origen y primeros enfoques

El BDSP es objeto de estudio en la literatura de investigación de operaciones desde la década de 1960 [3]. Ya en 1983, la revisión de Bodin et al. [1] ofrecía un panorama amplio de la programación de vehículos y tripulaciones, y los avances posteriores se difundieron sobre todo en la serie de conferencias *Computer-Aided Scheduling of Public Transport* (hoy CASPT) [3]. Wren y Rousseau [25] describen el problema y repasan los distintos enfoques de resolución disponibles a mediados de los noventa. Desde entonces, la literatura se ha bifurcado en dos grandes líneas: los métodos de programación matemática, en particular sobre modelos de partición y cobertura de conjuntos, y las heurísticas y metaheurísticas que trabajan directamente sobre los turnos [3].

3.2. Métodos exactos basados en partición y cobertura de conjuntos

La formulación más empleada es la de *set partitioning* (SPP) o *set covering* (SCP): cada columna de la matriz representa un turno factible y cada fila un tramo; se busca el subconjunto de columnas de costo mínimo que cubre cada fila exactamente una vez (SPP) o al menos una vez (SCP) [23, 17]. Su gran ventaja es que toda la complejidad de las reglas laborales queda encapsulada en la generación de columnas, de modo que el modelo es independiente de la legislación. Su gran desventaja es que el número de turnos posibles crece de manera exponencial con el número de tramos.

Para sortear esa dificultad se han seguido dos caminos. El primero es enumerar a priori un subconjunto acotado de turnos válidos y resolver el SCP sobre él, como en el sistema TRACS II, que además genera nuevas columnas a partir de ese conjunto enumerado, aunque solo en el nodo raíz del árbol de ramificación [11]. El segundo es la *generación de columnas* (CG), introducida para este problema por Desrochers y Soumis [6], que descompone el problema en un SCP maestro, que selecciona turnos entre los ya conocidos, y un subproblema de camino más corto, que genera nuevos turnos que mejoran la solución. Yunes et al. [27] combinaron CG con programación por restricciones para resolver instancias reales de hasta 246 tareas minimizando el número de conductores, y Chen y Shen [2] propusieron un algoritmo de CG mejorado aplicado a instancias de hasta 701 viajes. Curtis et al. [4] resolvieron un SPP restringido mediante programación por restricciones, usando la solución de la relajación lineal para guiar el orden de las variables y valores, lo que permite incorporar reglas no lineales. En el ámbito integrado, Huisman et al. [12]

resolvieron simultáneamente vehículos y conductores con múltiples depósitos mediante CG y relajación lagrangiana.

Como alternativa a SPP/SCP, De Leone et al. [17] propusieron un modelo compacto de programación entera en que las variables binarias indican qué tramo ocupa cada posición de cada turno, con restricciones explícitas de spreadover, tiempo de trabajo, cantidad de tiempos ociosos y retorno al depósito. Resuelto con CPLEX, el modelo encontró óptimos en instancias de hasta 25 tramos, pero en 70 tramos solo consiguió una primera solución factible para uno de los tres objetivos, lo que deja a la vista la limitación de los métodos exactos directos. Según la revisión de Constantino et al. [3], en general ningún trabajo había abordado instancias de más de 500 tareas mediante un enfoque exacto.

3.3. Metaheurísticas y heurísticas

Cuando las reglas son difíciles de expresar linealmente o las instancias son muy grandes, las metaheurísticas han sido la opción preferida. Las principales familias son:

Algoritmos genéticos. Wren y Wren [26] propusieron uno de los primeros algoritmos genéticos para el problema, aplicado sobre la formulación de cobertura de conjuntos. Dias et al. [7] extendieron la formulación SPP/SCP con un algoritmo genético capaz de considerar varios criterios de calidad simultáneamente, y Li y Kwan [20] incorporaron lógica difusa para evaluar la calidad de los turnos dentro de un algoritmo genético, resolviendo instancias de hasta 613 tareas.

Búsqueda tabú. Shen y Kwan [24] desarrollaron HACS, una búsqueda tabú que opera directamente sobre los turnos, sin recurrir a la enumeración de columnas propia de SPP/SCP.

GRASP. Lourenço et al. [21] usaron GRASP [8] tanto como método de resolución independiente como procedimiento interno de una búsqueda tabú y de algoritmos genéticos multiobjetivo. Su construcción es voraz, basada en el costo de los turnos, y su búsqueda local usa un vecindario de intercambio simple (*1-exchange*); estas técnicas se incorporaron al sistema comercial GIST. De Leone et al. [17] construyen cada turno agregando tramos compatibles desde una lista restringida de candidatos (RCL) ordenada por el tiempo de espera, y mejoran la solución con tres vecindarios: *1-swap* (intercambio de dos tramos entre turnos), *variable neighborhood swap* (intercambio de sufijos de turnos, que permite fusionar turnos y reducir su número) y una generalización a ρ puntos de corte. En instancias reales de hasta 161 viajes, GRASP obtuvo en segundos soluciones comparables, y en varios casos mejores, que un método exacto basado en SCP con columnas truncadas, que requería desde minutos hasta horas. Sobre las instancias aleatorias de Huisman et al. [12], además, obtuvo en promedio menos conductores, aunque con tipos de turno distintos, por lo que la comparación no es directa. Los mismos autores propusieron variantes multiarreglo aleatorizadas [18], y la hibridación de GRASP con *rollout* permitió resolver instancias de 250 tareas [5, 3].

Búsqueda en vecindario variable e hiperheurísticas. Ma et al. [22] emplearon VNS en un caso real de Beijing con 501 tareas, y Li et al. [19] propusieron una hiperheurística basada en CG para instancias reales de 500 tareas.

Heurística de múltiples asignaciones. Constantino et al. [3] proponen GraphBDSP, un algoritmo determinista que representa los tramos como vértices de un grafo multipartito dirigido y acíclico, donde los arcos indican que un tramo puede seguir a otro. Los vértices se agrupan en capas de tramos mutuamente incompatibles, y cada turno es un camino en el grafo. En la fase constructiva, recorre las capas y en cada una resuelve un problema de asignación lineal entre los turnos parciales y los tramos de la capa. En la fase de mejora, corta todos los turnos en una misma capa y recombina las partes izquierdas con las derechas mediante un nuevo problema de asignación (lo que nunca empeora la solución), además de un procedimiento que intenta eliminar el sobretiempo de los turnos largos. La factibilidad se maneja asignando costo infinito a las combinaciones que violan alguna regla, mientras que los cambios de línea se tratan como una penalización en el costo. El algoritmo tiene un solo parámetro y resolvió una instancia

real de 2.313 tareas en aproximadamente cuatro minutos. Los autores compararon además tres funciones de costo y encontraron que guiar la búsqueda por el tiempo ocioso y el sobretiempo da mejores resultados que usar directamente el tiempo pagado.

3.4. Mejores resultados y tendencia actual

Por la diversidad de reglas e instancias, no existe un algoritmo que domine en todas las variantes. Para la variante estudiada en este informe, el mejor resultado reportado corresponde a GraphBDSP [3]: en diez instancias de entre 130 y 2.313 tareas, sus soluciones fueron entre 1,6 % y 12 % más baratas que las del SCP con columnas generadas heurísticamente, y quedaron entre 4 % y 16 % por sobre una cota inferior obtenida de un modelo relajado. En instancias de tamaño mediano, GRASP mostró ser competitivo frente al método exacto con una fracción de su tiempo de cómputo [17].

La tendencia actual apunta a hibridar métodos exactos con metaheurísticas y a disponer de instancias públicas para comparar. Kletzander y Musliu [15] formalizaron reglas austriacas con restricciones de pausa muy complejas, publicaron un conjunto de instancias basadas en casos reales y propusieron una metaheurística que mejoró significativamente las soluciones reales. Sobre ese mismo problema aplicaron luego un método exacto de *branch and price* [16], y finalmente lo integraron con búsqueda en vecindades grandes (LNS): la metaheurística destruye parte de la solución y la repara resolviendo el subproblema con CG o *branch and price*, reutilizando las columnas generadas en iteraciones anteriores [14]. Una línea análoga, con un solo procedimiento exacto de bajo costo, es la del propio GraphBDSP, que usa el problema de asignación como operador de mejora [3].

3.5. Síntesis comparativa

La Tabla 1 resume los trabajos más representativos, en orden cronológico. Se observa un crecimiento sostenido en el tamaño de las instancias abordadas, en particular por los métodos heurísticos.

Tabla 1: Resumen de enfoques representativos para el BDSP, basado en la revisión de Constantino et al. [3]. El tamaño se informa en tareas o viajes, según lo reportado por cada autor.

Referencia	Objetivo	Técnica	Instancia mayor
Li y Kwan 2003 [20]	Costo y conductores	Alg. genético difuso	613 tareas
Yunes et al. 2005 [27]	Conductores	CG + prog. por restricciones	246 tareas
Huisman et al. 2005 [12]	Costo	CG + rel. lagrangiana	653 viajes
De Leone et al. 2011 [17]	Costo o conductores	MIP, GRASP	161 viajes
Chen y Shen 2013 [2]	Conductores	SCP + CG	701 viajes
Li et al. 2015 [19]	Costo y conductores	Hiperheurística + CG	500 tareas
Ma et al. 2016 [22]	Costo	VNS	501 tareas
Constantino et al. 2017 [3]	Costo (tiempo pagado)	Asignaciones sucesivas	2.313 tareas

En cuanto a la **representación**, conviven tres enfoques: (i) conjuntos de turnos preenumerados (columnas), propio de SPP/SCP [23]; (ii) turnos como secuencias de tramos o caminos en un grafo de compatibilidad, usado por GRASP [17] y por GraphBDSP [3]; y (iii) asignación directa de cada tramo a un turno, como en el modelo compacto de De Leone et al. [17], que asigna cada tramo a un turno y a una posición dentro de él. En cuanto a los **movimientos**, destacan el intercambio de tramos entre turnos [21, 17], el intercambio de sufijos, que permite fusionar o dividir turnos [17], y la recombinación simultánea de todos los turnos mediante problemas de asignación [3]. Finalmente, el **manejo de restricciones** se realiza mayoritariamente descartando las soluciones infactibles, ya sea filtrando los turnos durante la generación de columnas o

asignándoles costo infinito [3]. Las penalizaciones se reservan para restricciones blandas, como los cambios de línea en [3].

4. Modelo Matemático

Se presentan dos modelos. El primero es la formulación clásica de partición de conjuntos, útil como marco general. El segundo es un modelo compacto de programación lineal entera mixta (MILP) propuesto para la variante estudiada; toma como base las reglas y la función de costo de Constantino et al. [3], quienes indican que no conocen una formulación lineal para ellas, y adapta ideas del modelo de De Leone et al. [17], reemplazando las variables por posición por variables de arco.

4.1. Modelo 1: partición de conjuntos

Siguiendo la formulación clásica del problema [23, 17], sea J el conjunto de tramos y $\mathcal{P}$ el conjunto de todos los turnos factibles, es decir, aquellos que cumplen todas las reglas de la Sección 2.3. Para cada $p \in \mathcal{P}$, c_p es su costo (tiempo pagado más penalizaciones) y $a_{jp} = 1$ si el turno p contiene el tramo j . Con $z_p \in \{0, 1\}$ igual a 1 si se usa el turno p :

$$\min \sum_{p \in \mathcal{P}} c_p z_p \quad (1)$$

$$\text{s.a.} \quad \sum_{p \in \mathcal{P}} a_{jp} z_p = 1 \quad \forall j \in J \quad (2)$$

$$z_p \in \{0, 1\} \quad \forall p \in \mathcal{P} \quad (3)$$

La restricción (2) garantiza que cada tramo sea cubierto exactamente una vez. Al reemplazar “=” por “ $\geq$ ” se obtiene el SCP. El espacio de búsqueda es $2^{|\mathcal{P}|}$, y $|\mathcal{P}|$ crece exponencialmente con $|J|$, lo que motiva el uso de generación de columnas o de enumeración parcial.

4.2. Modelo 2: MILP compacto para la variante estudiada

Conjuntos y parámetros.

- $J = \{1, \dots, n\}$: tramos. Para cada $j \in J$: instante de inicio s_j y término $e_j = s_j + d_j$ (minutos desde las 00:00, con d_j la duración), lugar de partida a_j y de término b_j , línea ℓ_j , y minutos de conducción nocturna $\nu_j = |[s_j, e_j] \cap \mathcal{N}|$, donde $\mathcal{N} = \bigcup_{d \geq 0} [1440d - 120, 1440d + 300)$ es el horario nocturno (22:00 a 05:00) expresado en minutos, de modo que también se consideran los tramos posteriores a la medianoche.
- $K = \{1, \dots, m\}$: conductores disponibles ($m \leq n$ es una cota superior, por ejemplo $m = n$).
- $A = \{(i, j) \in J \times J : e_i \leq s_j, b_i = a_j\}$: pares compatibles, en que el conductor que termina i puede iniciar j . Se agregan un nodo origen 0 y un sumidero $n+1$, con arcos $(0, j)$ y $(j, n+1)$ para todo j . Como $e_i \leq s_j$ y $d_j > 0$, el grafo es acíclico.
- $g_{ij} = s_j - e_i$: pausa entre i y j . $A^R = \{(i, j) \in A : g_{ij} \geq R^{\min}\}$: pausas que pueden ser descanso. $\bar{g}_{ij} = \min\{g_{ij}, R^{\text{pag}}\}$: porción no pagada si la pausa es el descanso.
- $\lambda_{ij} = 1$ si $\ell_i \neq \ell_j$ (cambio de línea) y 0 en otro caso.
- $T^{\max}, R^{\min}, R^{\text{pag}}, P^{\min}, O^{\max}, S^{\max}, f_N, \beta_N, \beta_O, p_L$: parámetros de la Sección 2.3. $M = \max_{j \in J} e_j$: constante de las restricciones disyuntivas, igual al término del último tramo del día.

Variables.

- $x_{jk} \in \{0, 1\}$: 1 si el tramo j es asignado al conductor k .
- $y_{ijk} \in \{0, 1\}$: 1 si el conductor k realiza el tramo j inmediatamente después de i (incluye arcos desde 0 y hacia $n+1$).
- $r_{ijk} \in \{0, 1\}$, $(i, j) \in A^R$: 1 si la pausa entre i y j es el descanso del conductor k .
- $u_k \in \{0, 1\}$: 1 si el conductor k es utilizado.
- $S_k, E_k \geq 0$: inicio y término del turno k .
- $W_k \geq 0$: tiempo trabajado del turno k , corregido por el factor nocturno.
- $O_k \geq 0$: sobretiempo del turno k .

Como supuestos de modelamiento se considera, en concordancia con [3], que cada turno tiene a lo sumo un descanso no pagado, que las demás pausas son tiempo ocioso pagado, que no se exige volver al punto de partida al final del turno, y que el recargo nocturno se aplica solo a los minutos de conducción, no a las pausas en horario nocturno.

Función objetivo.

$$\min \sum_{k \in K} \left(P^{\min} u_k + (1 + \beta_O) O_k + \beta_N f_N \sum_{j \in J} \nu_j x_{jk} \right) + p_L \sum_{k \in K} \sum_{(i,j) \in A} \lambda_{ij} y_{ijk} \quad (4)$$

El primer término es el tiempo pagado. Si el tiempo trabajado W_k no supera $P^{\min}$, se paga $P^{\min}$ (la diferencia es tiempo ocioso); si lo supera, se paga $P^{\min} + O_k = W_k$ más el recargo $\beta_O O_k$ del sobretiempo. A ello se suma el recargo nocturno sobre los minutos nocturnos corregidos. El segundo término penaliza los cambios de línea. Esta forma de pago reproduce los ejemplos numéricos de Constantino et al. [3]: un turno de 480 minutos trabajados se paga $432 + 48 \cdot 1,5 = 504$ minutos, y uno de 420 minutos se paga 432.

Restricciones.

$$\sum_{k \in K} x_{jk} = 1 \quad \forall j \in J \quad (5)$$

$$\sum_{i: (i,j) \in A \cup \{(0,j)\}} y_{ijk} = x_{jk}, \quad \sum_{l: (j,l) \in A \cup \{(j,n+1)\}} y_{jlk} = x_{jk} \quad \forall j \in J, k \in K \quad (6)$$

$$\sum_{j \in J} y_{0jk} = u_k, \quad \sum_{j \in J} y_{j,n+1,k} = u_k \quad \forall k \in K \quad (7)$$

$$S_k = \sum_{j \in J} s_j y_{0jk}, \quad E_k = \sum_{j \in J} e_j y_{j,n+1,k} \quad \forall k \in K \quad (8)$$

$$E_k - S_k \leq S^{\max} \quad \forall k \in K \quad (9)$$

$$r_{ijk} \leq y_{ijk} \quad \forall (i,j) \in A^R, k \in K \quad (10)$$

$$\sum_{(i,j) \in A^R} r_{ijk} \leq 1 \quad \forall k \in K \quad (11)$$

$$E_k - S_k \geq (T^{\max} + 1) \sum_{(i,j) \in A^R} r_{ijk} \quad \forall k \in K \quad (12)$$

$$E_k - S_k \leq T^{\text{máx}} + M \sum_{(i,j) \in A^R} r_{ijk} \quad \forall k \in K \quad (13)$$

$$e_i - S_k \leq T^{\text{máx}} + M(1 - r_{ijk}), \quad E_k - s_j \leq T^{\text{máx}} + M(1 - r_{ijk}) \quad \forall (i,j) \in A^R, k \in K \quad (14)$$

$$W_k = E_k - S_k - \sum_{(i,j) \in A^R} \bar{g}_{ij} r_{ijk} + (f_N - 1) \sum_{j \in J} \nu_j x_{jk} \quad \forall k \in K \quad (15)$$

$$O_k \geq W_k - P^{\text{mín}} u_k \quad \forall k \in K \quad (16)$$

$$O_k \leq O^{\text{máx}} \quad \forall k \in K \quad (17)$$

$$u_k \geq u_{k+1} \quad k = 1, \dots, m-1 \quad (18)$$

Explicación. (5) asegura que cada tramo sea realizado por exactamente un conductor, de modo que ningún tramo quede sin cubrir ni sea compartido. (6) enlaza la asignación con la secuencia: cada tramo asignado a k tiene exactamente un antecesor y un sucesor en el turno de k (posiblemente el origen o el sumidero). Como los arcos solo existen entre tramos compatibles, esto impone que un conductor no inicie un tramo antes de terminar el anterior ni en un lugar distinto de aquel en que se encuentra. (7) hace que cada conductor utilizado tenga un único camino desde el origen al sumidero; al ser el grafo acíclico, no se requieren restricciones de eliminación de subtours. (8) define el inicio y el término del turno, que valen 0 para conductores no utilizados, y (9) limita la duración total del turno (regla 6). (10) y (11) permiten designar como descanso a lo más una pausa del turno, de largo al menos $R^{\text{mín}}$, y (12) solo lo permite si el turno dura más que $T^{\text{máx}}$; en caso contrario, todas sus pausas son pagadas, como en [3]. (13) y (14) modelan la regla 1: si el turno no tiene descanso, su duración total no puede exceder $T^{\text{máx}}$; si lo tiene, tanto el tramo de trabajo anterior como el posterior al descanso deben durar a lo más $T^{\text{máx}}$. (15) calcula el tiempo trabajado como el spreadover menos la parte no pagada del descanso (regla 2), considerando que los minutos nocturnos valen f_N veces (regla 5). (16) define el sobretiempo como el exceso sobre la jornada; al minimizar, $O_k = \text{máx}\{0, W_k - P^{\text{mín}}\}$ y así el primer término de (4) implementa la regla 3. (17) limita el sobretiempo (regla 4). Por último, (18) elimina simetrías entre conductores equivalentes, obligando a usar primero los de menor índice.

Espacio de búsqueda. Dado que dentro de un turno el orden de los tramos queda determinado por sus horarios, una solución queda completamente definida por las variables x_{jk} ; las variables y , r y las continuas se derivan de ellas. Por lo tanto, el espacio de búsqueda tiene tamaño $|\Omega| \leq m^n$ (o a lo más el número de Bell B_n si se ignora la identidad de los conductores), del cual solo una fracción pequeña es factible. El modelo tiene $O(|A|m) = O(n^2m)$ variables binarias y restricciones, lo que, al igual que el modelo de De Leone et al. [17], lo hace práctico solo para instancias pequeñas. Sin embargo, sirve para definir de manera precisa la función de evaluación y las restricciones que deberá manejar un algoritmo de búsqueda.

5. Conclusiones

El estudio muestra que el BDSP no es un problema único, sino una familia de problemas cuyas diferencias provienen principalmente de las reglas laborales de cada país y empresa y del objetivo considerado (costo, número de conductores o ambos). Por ello, las técnicas no resuelven exactamente el mismo problema, y sus resultados son difíciles de comparar: varios trabajos se evalúan con datos de una empresa específica, como los de PluService en [17] o los de una empresa brasileña en [3], y solo recientemente se han publicado conjuntos de instancias de referencia [3, 15].

Las técnicas se agrupan en dos grandes enfoques. Los métodos basados en SPP/SCP y generación de columnas separan limpiamente las reglas (en la generación de turnos) de la optimización y entregan cotas de calidad, pero su costo computacional crece rápidamente, y cuando las columnas se generan de forma heurística pierden la garantía de optimalidad: en [3], el SCP con columnas truncadas fue superado por una heurística. Las metaheurísticas (GRASP, tabú, genéticos, VNS) y las heurísticas especializadas abordan instancias más grandes, en general de hasta unas 600 tareas y, en el caso de GraphBDSP, de más de 2.000 [3], pero no garantizan optimalidad y dependen fuertemente del diseño de sus movimientos. Las dos fuentes base del curso ilustran bien este contraste dentro de las heurísticas: el GRASP de De Leone et al. [17] es estocástico y multiarranque, por lo que su resultado depende de la aleatorización y del número de iteraciones, mientras que GraphBDSP [3] es determinista, tiene un solo parámetro y mejora la solución resolviendo problemas de asignación de forma exacta. En ambos, las reglas de descanso y de pago dependen del turno completo y no son lineales, por lo que los turnos se evalúan mediante una función procedimental y se descartan los infactibles [17, 3].

En la variante estudiada, el tiempo pagado mínimo cumple un papel central: todo turno se paga por al menos 432 minutos, de modo que los turnos cortos generan un gran tiempo ocioso pagado. En el ejemplo del curso, dos turnos de menos de 200 minutos de trabajo cuestan 864 minutos. En consecuencia, las estrategias más prometedoras son: (i) los movimientos que operan sobre segmentos de turnos, como el intercambio de sufijos [17] y la recombinación por asignación [3], porque permiten fusionar turnos cortos y reducir el tiempo ocioso; (ii) guiar la búsqueda con componentes específicos del costo, como el tiempo ocioso y el sobretiempo, en lugar del tiempo pagado total, lo que dio mejores resultados en [3]; y (iii) los híbridos entre métodos exactos y metaheurísticas, como LNS con reparación por *branch and price* [14], que representan la tendencia actual.

El modelo propuesto en la Sección 4 tiene limitaciones: por su tamaño, $O(n^2m)$, solo es práctico para instancias pequeñas, y supone un único descanso por turno y que el conductor no necesita volver al depósito. Aun así, entrega una definición precisa de la función de evaluación y de las restricciones que deberá manejar la implementación de la siguiente entrega. Como trabajo futuro, se propone explorar representaciones de asignación directa tramo-conductor con evaluación incremental del costo, que permiten movimientos baratos y un manejo explícito de las restricciones, ya sea descartando los vecinos infactibles o penalizándolos. También se propone relajar los supuestos del modelo (varios descansos, retorno al depósito), estudiar la reprogramación dinámica ante eventos imprevistos, señalada como ventaja potencial por [3], y la integración con la etapa de *crew rostering*.

6. Uso LLMs

1. **Herramienta utilizada:** Claude Code (interfaz de línea de comandos de Anthropic), con el modelo Claude Opus 5.5.
2. **Propósito de uso:** búsqueda exploratoria de literatura sobre BDSP, identificación de autores y trabajos relevantes, redacción de borradores iniciales para algunas secciones, corrección de estilo y ortografía.
3. **Prompts relevantes:** “Resumen del estado del arte de Bus Driver Scheduling Problem desde Weaver (1972) hasta heurísticas modernas”; “Formulación clásica de Set Covering (o Set Partitioning) binario puro generado en dos etapas”; “Referencias académicas relevantes”.
4. **Nivel de edición:** el contenido entregado por la herramienta fue reescrito, reorganizado y verificado por el autor. Las referencias fueron consultadas individualmente en sus fuentes originales para confirmar autoría, año de publicación y exactitud de las afirmaciones citadas.

5. **Validación:** el contenido se contrastó con los artículos fuente [17, 3] y con la presentación del problema entregada por el ayudante. Los datos bibliográficos de las referencias adicionales se verificaron mediante búsquedas en las páginas de las editoriales (AAAI, JAIR, Elsevier, Taylor & Francis).

Referencias

- [1] Lawrence Bodin, Bruce Golden, Arjang Assad, and Michael Ball. Routing and scheduling of vehicles and crews: the state of the art. *Computers & Operations Research*, 10(2):63–212, 1983.
- [2] Shijun Chen and Yindong Shen. An improved column generation algorithm for crew scheduling problems. *Journal of Information and Computational Science*, 10(1):175–183, 2013.
- [3] Ademir Aparecido Constantino, Candido Ferreira Xavier de Mendonça Neto, Silvio Alexandre de Araujo, Dario Landa-Silva, Rogério Calvi, and Allainclair Flausino dos Santos. Solving a large real-world bus driver scheduling problem with a multi-assignment based heuristic algorithm. *Journal of Universal Computer Science*, 23(5):479–504, 2017.
- [4] Suniel D. Curtis, Barbara M. Smith, and Anthony Wren. Forming bus driver schedules using constraint programming. Technical Report Report 99.05, University of Leeds, School of Computer Studies, 1999.
- [5] Gianluca D’Annibale, Renato De Leone, Paola Festa, and Emilia Marchitto. A new meta-heuristic for the bus driver scheduling problem: GRASP combined with rollout. In *2007 IEEE Symposium on Computational Intelligence in Scheduling (CISched 2007)*, pages 192–197, 2007.
- [6] Martin Desrochers and François Soumis. A column generation approach to the urban transit crew scheduling problem. *Transportation Science*, 23(1):1–13, 1989.
- [7] Teresa Galvão Dias, Jorge Pinho de Sousa, and João Falcão e Cunha. Genetic algorithms for the bus driver scheduling problem: a case study. *Journal of the Operational Research Society*, 53(3):324–335, 2002.
- [8] Thomas A. Feo and Mauricio G. C. Resende. Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6:109–133, 1995.
- [9] Matteo Fischetti, Silvano Martello, and Paolo Toth. The fixed job schedule problem with spread-time constraints. *Operations Research*, 35(6):849–858, 1987.
- [10] Matteo Fischetti, Silvano Martello, and Paolo Toth. The fixed job schedule problem with working-time constraints. *Operations Research*, 37(3):395–403, 1989.
- [11] Sarah Fores, Les Proll, and Anthony Wren. TRACS II: a hybrid IP/heuristic driver scheduling system for public transport. *Journal of the Operational Research Society*, 53(10):1093–1100, 2002.
- [12] Dennis Huisman, Richard Freling, and Albert P. M. Wagelmans. Multiple-depot integrated vehicle and crew scheduling. *Transportation Science*, 39(4):491–502, 2005.
- [13] Omar J. Ibarra-Rojas, Felipe Delgado, Ricardo Giesen, and Juan Carlos Muñoz. Planning, operation, and control of bus transport systems: A literature review. *Transportation Research Part B: Methodological*, 77:38–75, 2015.

- [14] Lucas Kletzander, Tommaso Mannelli Mazzoli, Nysret Musliu, and Pascal Van Hentenryck. Integrating column generation and large neighborhood search for bus driver scheduling with complex break constraints. *Journal of Artificial Intelligence Research*, 85:44:1–44:37, 2026. Preprint: arXiv:2505.02485.
- [15] Lucas Kletzander and Nysret Musliu. Solving large real-life bus driver scheduling problems with complex break constraints. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, volume 30, pages 421–429, 2020. DOI: 10.1609/icaps.v30i1.6688.
- [16] Lucas Kletzander, Nysret Musliu, and Pascal Van Hentenryck. Branch and price for bus driver scheduling with complex break constraints. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 11853–11861, 2021.
- [17] Renato De Leone, Paola Festa, and Emilia Marchitto. A bus driver scheduling problem: a new mathematical model and a GRASP approximate solution. *Journal of Heuristics*, 17(4):441–466, 2011. DOI: 10.1007/s10732-010-9141-3.
- [18] Renato De Leone, Paola Festa, and Emilia Marchitto. Solving a bus driver scheduling problem with randomized multistart heuristics. *International Transactions in Operational Research*, 18(6):707–727, 2011.
- [19] Huaxin Li, Yu Wang, Shaohua Li, and Shuo Li. A column generation based hyper-heuristic to the bus driver scheduling problem. *Discrete Dynamics in Nature and Society*, 2015:1–10, 2015.
- [20] Jingpeng Li and Raymond S. K. Kwan. A fuzzy genetic algorithm for driver scheduling. *European Journal of Operational Research*, 147(2):334–344, 2003.
- [21] Helena R. Lourenço, José P. Paixão, and Rita Portugal. Multiobjective metaheuristics for the bus-driver scheduling problem. *Transportation Science*, 35(3):331–343, 2001.
- [22] Jihui Ma, Avishai Ceder, Yang Yang, Tao Liu, and Wei Guan. A case study of Beijing bus crew scheduling: a variable neighborhood-based approach. *Journal of Advanced Transportation*, 50(4):434–445, 2016.
- [23] Rita Portugal, Helena R. Lourenço, and José P. Paixão. Driver scheduling problem modelling. *Public Transport*, 1(2):103–120, 2009.
- [24] Yindong Shen and Raymond S. K. Kwan. Tabu search for driver scheduling. In S. Voß and J. R. Daduna, editors, *Computer-Aided Scheduling of Public Transport*, volume 505 of *Lecture Notes in Economics and Mathematical Systems*, pages 121–135. Springer, Berlin, Heidelberg, 2001.
- [25] Anthony Wren and Jean-Marc Rousseau. Bus driver scheduling – an overview. In J. R. Daduna, I. Branco, and J. M. P. Paixão, editors, *Computer-Aided Transit Scheduling*, volume 430 of *Lecture Notes in Economics and Mathematical Systems*, pages 173–187. Springer, Berlin, Heidelberg, 1995.
- [26] Anthony Wren and Douglas O. Wren. A genetic algorithm for public transport driver scheduling. *Computers & Operations Research*, 22(1):101–110, 1995.
- [27] Tallys H. Yunes, Arnaldo V. Moura, and Cid C. de Souza. Hybrid column generation approaches for urban transit crew management problems. *Transportation Science*, 39(2):273–288, 2005.